

Merge Sort Algorithm: A Structured Note

1 The Divide and Conquer Paradigm

Merge Sort is a sorting algorithm that applies the “divide and conquer” design principle. The basic premise is to break the input array down into parts, solve the problem for each part, and combine the solutions. Specifically, the Merge Sort algorithm consists of three steps:

- **Divide:** The input array is divided into two equal halves. This division step is trivial and takes virtually no time.
- **Sort:** Each of the two pieces is sorted using recursion.
- **Put Together:** The final step involves taking the two sorted arrays and combining them into a single sorted array.

2 The Crucial Step: Merging

While dividing takes no effort, the most critical part of Merge Sort is the step where the two sorted arrays are put together, which is called “merging”. The procedure to build this bigger, sorted array works as follows:

- The algorithm utilizes two pointers (or temporary variables) that point to positions within the two smaller sorted arrays, let's call them A and B .
- It compares the elements pointed to in both arrays to find the smallest one. For example, if the pointer in array A is a and the pointer in array B is b , it checks if $A[a] < B[b]$.
- The smaller of the two elements is placed into the first available position in a new, larger array, let's call it C .
- The pointers for the array that supplied the element and the new array C are then incremented (e.g., $a++$ and $c++$). This scanning process continues until all elements are filled into the big array C .

3 Calculating the Comparisons for the Merge Step

To determine the time complexity, it is necessary to figure out how much time this merging procedure takes for an array of size n . The notes provide two methods for calculating the number of comparisons required:

- **The First Way (Observation):** In the “smart way” of observing the algorithm, every time a comparison is made, exactly one element gets filled into the new array C . Because the total number of elements that need to be placed in array C is n , the total number of comparisons is bounded by n .

- **The Second Way (Induction/Recursion):** This method builds larger arrays from smaller ones. After the first comparison is made and the smaller element is placed into array C , the remaining problem size decreases by one. If n is the sum of the elements in the two subarrays, the time to merge can be written as the recurrence relation $T(n) = T(n - 1) + 1$ (where the "+1" represents the first comparison). Given the base case of $T(2) = 1$, solving this recurrence reveals that the exact number of comparisons needed is $n - 1$.

4 Overall Time Complexity Analysis

Since any other operations (like moving the pointers) are bounded by the number of comparisons, the algorithm's running time focuses primarily on comparisons.

- The total time $T(n)$ for Merge Sort is the sum of the time taken to sort the two halves plus the linear time $\mathcal{O}(n)$ taken by the merge step.
- This forms the recurrence relation:

$$T(n) = 2T(n/2) + n$$

- To solve this, the recurrence is repeatedly expanded:

$$T(n) = 2(2T(n/2^2) + n/2) + n = 2^2T(n/2^2) + 2n$$

- If expanded i times, the formula becomes:

$$T(n) = 2^iT(n/2^i) + in$$

- The recursion terminates at a base case, such as when the subarray size is 2 (i.e., $n/2^i = 2$), which means $i = \log_2(n/2)$.
- By substituting i back into the formula, we find that the overall time taken by the Merge Sort algorithm evaluates to roughly $\mathcal{O}(n \log n)$.